

# COMP9319 Web Data Compression and Search

LZW,  
Adaptive Huffman,  
(live lecture)

1

## Note: LZW decoding

- There is one special case that the LZW decoding pseudocode presented is unable to handle.
- This is your exercise to find out in what situation that happens, and how to deal with it.
- I'll go through this at the live lecture.

2

## LZW Notes (Handling the special case)

3

### Encoding

Input: abababab



|     |   | Dict [index] |              |
|-----|---|--------------|--------------|
| p   | c | output       | index symbol |
| nil | a |              |              |

```
p = nil; // p for prev char
while read(c);
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

4

### Encoding

Input: abababab



|     |   | Dict [index] |              |
|-----|---|--------------|--------------|
| p   | c | output       | index symbol |
| nil | a |              |              |
| a   | b | a            | 256 ab       |

```
p = nil; // p for prev char
while read(c);
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

5

### Encoding

Input: abababab



|     |   | Dict [index] |              |
|-----|---|--------------|--------------|
| p   | c | output       | index symbol |
| nil | a |              |              |
| a   | b | a            | 256 ab       |
| b   | a | b            | 257 ba       |

```
p = nil; // p for prev char
while read(c);
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

6

## Encoding

Input: abababab  
↑

```
p = nil; // p for prev char
while read(c):
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

|     |   | Dict [index] |       |        |
|-----|---|--------------|-------|--------|
| p   | c | output       | index | symbol |
| nil | a |              |       |        |
| a   | b | a            | 256   | ab     |
| b   | a | b            | 257   | ba     |
| a   | b |              |       |        |

7

## Encoding

Input: abababab  
↑

```
p = nil; // p for prev char
while read(c):
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

|     |   | Dict [index] |       |        |
|-----|---|--------------|-------|--------|
| p   | c | output       | index | symbol |
| nil | a |              |       |        |
| a   | b | a            | 256   | ab     |
| b   | a | b            | 257   | ba     |
| a   | b |              |       |        |
| ab  | a | 256          | 258   | aba    |

8

## Encoding

Input: abababab  
↑

```
p = nil; // p for prev char
while read(c):
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

|     |   | Dict [index] |       |        |
|-----|---|--------------|-------|--------|
| p   | c | output       | index | symbol |
| nil | a |              |       |        |
| a   | b | a            | 256   | ab     |
| b   | a | b            | 257   | ba     |
| a   | b |              |       |        |
| ab  | a | 256          | 258   | aba    |
| a   | b |              |       |        |

9

## Encoding

Input: abababab  
↑

```
p = nil; // p for prev char
while read(c):
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

|     |   | Dict [index] |       |        |
|-----|---|--------------|-------|--------|
| p   | c | output       | index | symbol |
| nil | a |              |       |        |
| a   | b | a            | 256   | ab     |
| b   | a | b            | 257   | ba     |
| a   | b |              |       |        |
| ab  | a | 256          | 258   | aba    |
| a   | b |              |       |        |
| ab  | a |              |       |        |

10

## Encoding

Input: abababab  
↑

```
p = nil; // p for prev char
while read(c):
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

|     |   | Dict [index] |       |        |
|-----|---|--------------|-------|--------|
| p   | c | output       | index | symbol |
| nil | a |              |       |        |
| a   | b | a            | 256   | ab     |
| b   | a | b            | 257   | ba     |
| a   | b |              |       |        |
| ab  | a | 256          | 258   | aba    |
| a   | b |              |       |        |
| ab  | a |              |       |        |
| aba | b | 258          | 259   | abab   |

11

## Encoding

Input: abababab  
↑

```
p = nil; // p for prev char
while read(c):
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

|     |     | Dict [index] |       |        |
|-----|-----|--------------|-------|--------|
| p   | c   | output       | index | symbol |
| nil | a   |              |       |        |
| a   | b   | a            | 256   | ab     |
| b   | a   | b            | 257   | ba     |
| a   | b   |              |       |        |
| ab  | a   | 256          | 258   | aba    |
| a   | b   |              |       |        |
| ab  | a   |              |       |        |
| aba | b   | 258          | 259   | abab   |
| b   | EOF | b            |       |        |

12

## Encoding

Input: abababab

```
p = nil; // p for prev char
while read(c):
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

|     |     |            | Dict [index] |        |
|-----|-----|------------|--------------|--------|
| p   | c   | output     | index        | symbol |
| nil | a   |            |              |        |
| a   | b   | <b>a</b>   | 256          | ab     |
| b   | a   | <b>b</b>   | 257          | ba     |
| a   | b   |            |              |        |
| ab  | a   | <b>256</b> | 258          | aba    |
| a   | b   |            |              |        |
| ab  | a   |            |              |        |
| aba | b   | <b>258</b> | 259          | abab   |
| b   | EOF | <b>b</b>   |              |        |

13

## Decoding

Input: ab<256><258>b

```
read(c); // c is likely to be > 8 bits
output c;
p = c;
while read(c):
  output Dict[c];
  add p + Dict[c][0] to Dict;
  p = Dict[c];
```

|   |   |        | Dict [index] |        |
|---|---|--------|--------------|--------|
| p | c | output | index        | symbol |
|   | a | a      |              |        |

14

## Decoding

Input: ab<256><258>b

```
read(c); // c is likely to be > 8 bits
output c;
p = c;
while read(c):
  output Dict[c];
  add p + Dict[c][0] to Dict;
  p = Dict[c];
```

|   |   |        | Dict [index] |        |
|---|---|--------|--------------|--------|
| p | c | output | index        | symbol |
|   | a | a      |              |        |
| a | b | b      | 256          | ab     |

15

## Decoding

Input: ab<256><258>b

```
read(c); // c is likely to be > 8 bits
output c;
p = c;
while read(c):
  output Dict[c];
  add p + Dict[c][0] to Dict;
  p = Dict[c];
```

|   |       |        | Dict [index] |        |
|---|-------|--------|--------------|--------|
| p | c     | output | index        | symbol |
|   | a     | a      |              |        |
| a | b     | b      | 256          | ab     |
| b | <256> | ab     | 257          | ba     |

16

## Decoding

Input: ab<256><258>b

```
read(c); // c is likely to be > 8 bits
output c;
p = c;
while read(c):
  output Dict[c];
  add p + Dict[c][0] to Dict;
  p = Dict[c];
```

|       |       |           | Dict [index] |        |
|-------|-------|-----------|--------------|--------|
| p     | c     | output    | index        | symbol |
|       | a     | a         |              |        |
| a     | b     | b         | 256          | ab     |
| b     | <256> | ab        | 257          | ba     |
| <256> | <258> | ← WHAT??? |              |        |

17

## Encoding

Input: abababab

```
p = nil; // p for prev char
while read(c):
  if pc ∈ Dict:
    p = pc;
  else:
    add pc to Dict;
    output code(p);
    p = c;
```

|            |          |            | Dict [index] |            |
|------------|----------|------------|--------------|------------|
| p          | c        | output     | index        | symbol     |
| nil        | a        |            |              |            |
| a          | b        | a          | 256          | ab         |
| b          | a        | b          | 257          | ba         |
| a          | b        |            |              |            |
| <b>ab</b>  | <b>a</b> | 256        | <b>258</b>   | <b>aba</b> |
| a          | b        |            |              |            |
| ab         | a        |            |              |            |
| <b>aba</b> | <b>b</b> | <b>258</b> | 259          | abab       |
| b          | EOF      | b          |              |            |

18

## Decoding

Input: ab<256><258>b

```
read(c); // c is likely to be > 8 bits
output c;
p = c;
while read(c):
    output Dict[c];
    add p + Dict[c][0] to Dict;
    p = Dict[c];
```

|       |       | Dict [index] |        |
|-------|-------|--------------|--------|
| p     | c     | index        | symbol |
| a     | a     |              |        |
| a     | b     | 256          | ab     |
| b     | <256> | 257          | ba     |
| <256> | <258> | 258          | aba    |

19

## Decoding

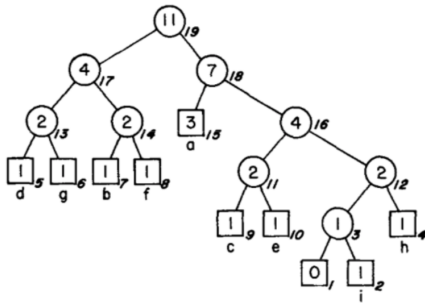
Input: ab<256><258>b

```
read(c); // c is likely to be > 8 bits
output c;
p = c;
while read(c):
    output Dict[c];
    add p + Dict[c][0] to Dict;
    p = Dict[c];
```

|       |       | Dict [index] |        |
|-------|-------|--------------|--------|
| p     | c     | index        | symbol |
| a     | a     |              |        |
| a     | b     | 256          | ab     |
| b     | <256> | 257          | ba     |
| <256> | <258> | 258          | aba    |
| <258> | b     | 259          | abab   |

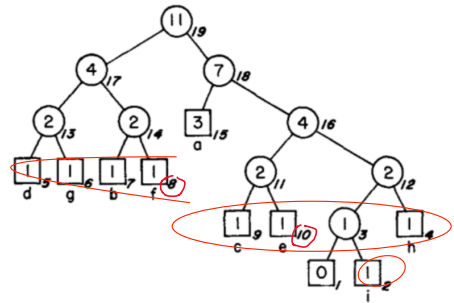
20

## Adaptive Huffman (FGK)



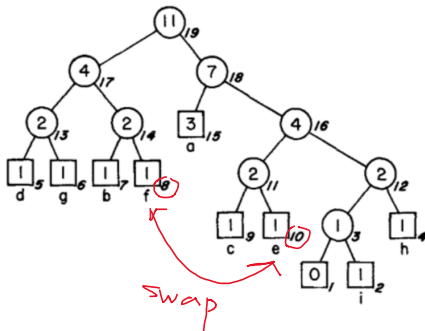
22

## Adaptive Huffman (FGK)



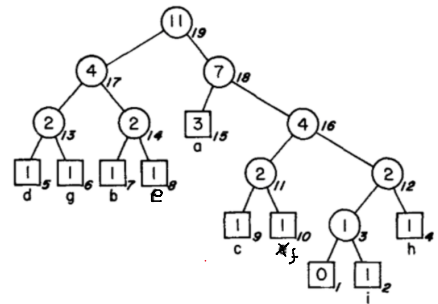
23

## Adaptive Huffman (FGK)



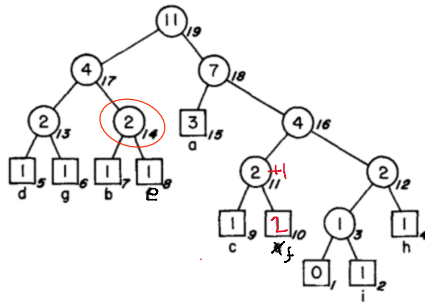
24

## Adaptive Huffman (FGK)



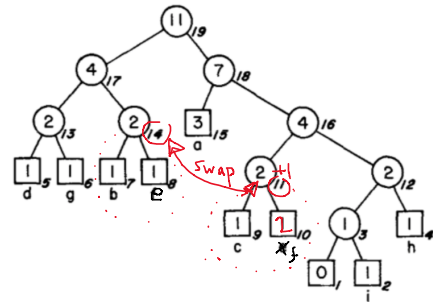
25

## Adaptive Huffman (FGK)



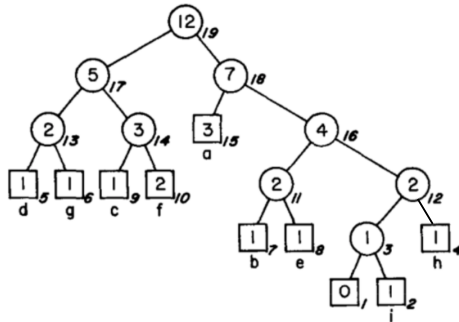
26

## Adaptive Huffman (FGK)



27

## Adaptive Huffman (FGK): when f is inserted



28

## Adaptive Huffman (FGK vs Vitter)

1. FGK: (Explicit) node numbering  
Vitter: Implicit numbering

2. Vitter's Invariant:

- (\*) For each weight  $w$ , all leaves of weight  $w$  precede (in the implicit numbering) all internal nodes of weight  $w$ .

In order to maintain the invariant (\*), we must keep separate blocks for internal and leaf nodes.

49

## Adaptive Huffman (Vitter'87)

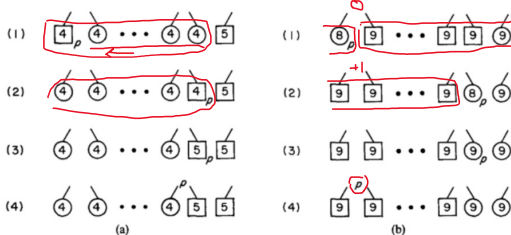


FIG. 6. Algorithm A's SlideAndIncrement operation. All the nodes in a given block shift to the left one spot to make room for node  $p$ , which slides over the block to the right. (a) Node  $p$  is a leaf of weight 4. The internal nodes of weight 4 shift to the left. (b) Node  $p$  is an internal node of weight 8. The leaves of weight 9 shift to the left.

50

## The Vitter's algo: swaps then slides

**Definition 4.1.** Blocks are equivalence classes of nodes defined by  $v = x$  iff nodes  $v$  and  $x$  have the same weight and are either both internal nodes or both leaves. The leader of a block is the highest numbered (in the implicit numbering) node in a block.

The blocks are linked together by increasing order of weight; a leaf block always precedes an internal block of the same weight. The main operation of the algorithm needed to maintain invariant (\*) is the SlideAndIncrement operation, illustrated in Figure 6. The version of Update we use for Algorithm A is outlined below:

```

procedure Update;
begin
  leafTolIncrement := 0;
  q := leaf node corresponding to  $a_{n-1}$ ;
  if (q is the 0-node) and (i < n - 1) then
    begin (Special Case #1)
      Replace q by an internal 0-node with two leaf 0-node children, such that the right child
      corresponds to  $a_{n-1}$ ;
      q := internal 0-node just created;
      leafTolIncrement := the right child of q
    end
  else begin
    Interchange q in the tree with the leader of its block;
    if q is the sibling of the 0-node then
      begin (Special Case #2)
        leafTolIncrement := q;
        q := parent of q
      end
    end
  while q is not the root of the Huffman tree do
    (Main loop; q must be the leader of its block)
    SlideAndIncrement(q);
    if leafTolIncrement ≠ 0 then (Handle the two special cases)
      SlideAndIncrement(leafTolIncrement);
  end;

```

Add a new node

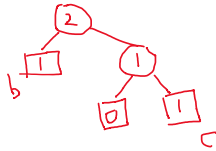
Update the node next to NYT

58

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string bcaaabbb.



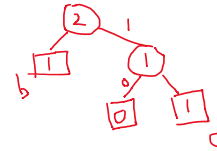
01100010 001100011

20

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string bcaaabbb.



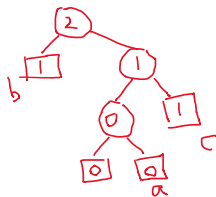
01100010 001100011 1001100001

21

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string bcaaabbb.



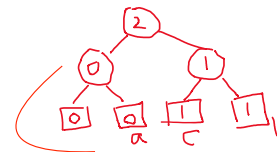
01100010 001100011 1001100001

22

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string bcaaabbb.



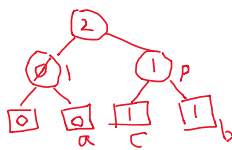
01100010 001100011 1001100001

23

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string bcaaabbb.



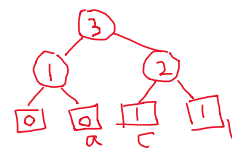
01100010 001100011 1001100001

24

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string bcaaabbb.



01100010 001100011 1001100001

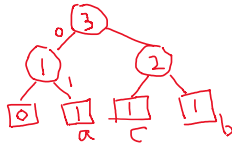
25

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaaabbb**.

z



01100010 001100011 1001100001 01

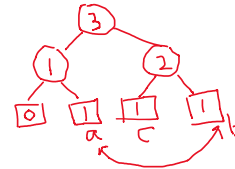
26

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaaabbb**.

z



01100010 001100011 1001100001 01

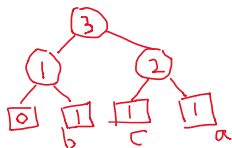
27

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaaabbb**.

z



01100010 001100011 1001100001 01

28

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaaabbb**.

z



01100010 001100011 1001100001 01

29

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaaabbb**.

z



01100010 001100011 1001100001 01

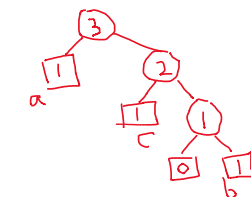
30

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaaabbb**.

z



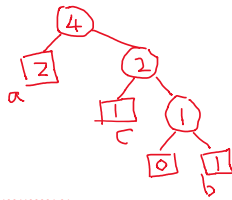
01100010 001100011 1001100001 01

31

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



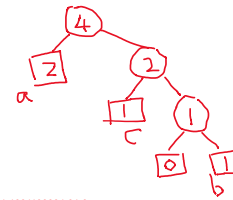
01100010 001100011 1001100001 01

32

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



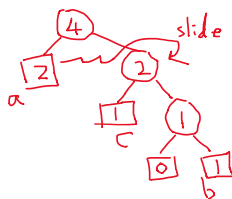
01100010 001100011 1001100001 01 0

33

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



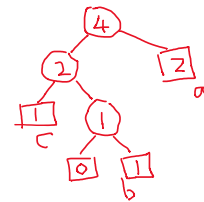
01100010 001100011 1001100001 01 0

34

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



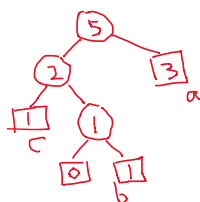
01100010 001100011 1001100001 01 0

35

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



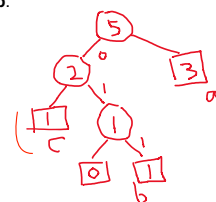
01100010 001100011 1001100001 01 0

36

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



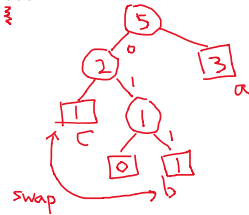
01100010 001100011 1001100001 01 0 011

37

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



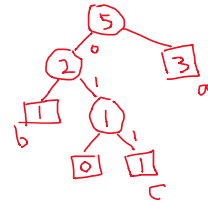
38

01100010 001100011 1001100001 01 0 011

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



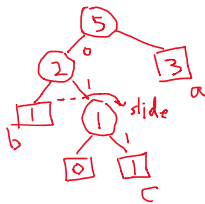
39

01100010 001100011 1001100001 01 0 011

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



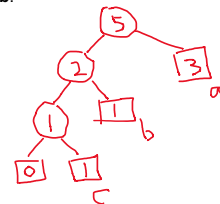
40

01100010 001100011 1001100001 01 0 011

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



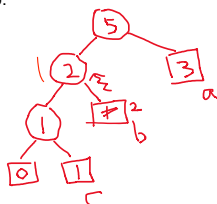
41

01100010 001100011 1001100001 01 0 011

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



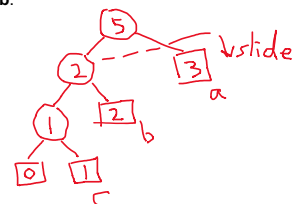
42

01100010 001100011 1001100001 01 0 011

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



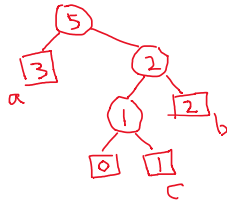
43

01100010 001100011 1001100001 01 0 011

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001, b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string **bcaaabbb**.



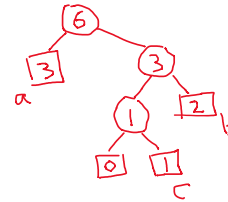
44

01100010 001100011 1001100001 01 0 011

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001, b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string **bcaaabbb**.



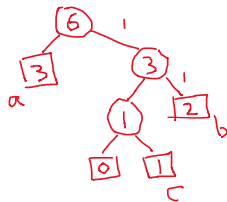
45

01100010 001100011 1001100001 01 0 011

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001, b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string **bcaaabbb**.



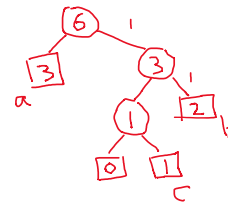
46

01100010 001100011 1001100001 01 0 011

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001, b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string **bcaaabbb**.



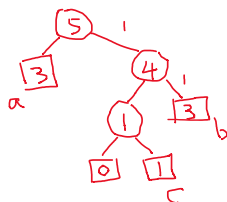
47

01100010 001100011 1001100001 01 0 011 11

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001, b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string **bcaaabbb**.



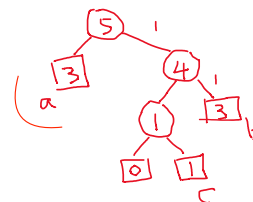
48

01100010 001100011 1001100001 01 0 011 11

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001, b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the string **bcaaabbb**.



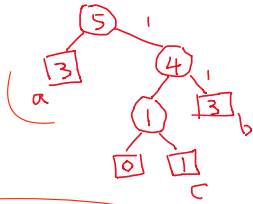
49

01100010 001100011 1001100001 01 0 011 11 11

## Adaptive Huffman (Ex2, Q2)

The initial coding before any transmission is: a=01100001,  
b=01100010, c=01100011.

Derive the encoded bitstream produced by the encoder for the  
string **bcaaabbb**.



01100010 001100011 1001100001 01 0 011 11 11